Relatório 4 - Prática: Principais Bibliotecas e Ferramentas Python para Aprendizado de Máquina (I)

Eduardo Andrei Staudt

1. Introdução

O objetivo deste card foi proporcionar uma ambientação das ferramentas utilizadas em ML(*Machine Learning*) e IA(Inteligência Artificial). No decorrer do curso desenvolvi habilidades e a aprendi a mexer em ferramentas como *Jupyter Notebook*, *Numpy* e *Pandas*, que são as principais ferramentas de manipulação de dados e ambientes de treinamento e construção de algoritmos de ML e IA

2. Desenvolvimento

Inicialmente comecei pela seção 3 na primeira aula sobre *Jupyter Notebook* fiz a instalação do mesmo com `pip install notebook` no terminal e como eu não possuía o anaconda tive que adicionar o `/Script` na *PATH* para poder rodar como comando utilizado no curso `jupyter notebook`. Depois disso comecei a ver sobre como funciona mesmo a ferramenta e alguns atalhos, aprendi sobre as células de python e de markdown e como elas são muito úteis.

Na segunda aula aprendi sobre ambientes virtuais que me permitem utilizar o virtualenv para utilizar diversas versões de ferramentas e tecnologias sem perder a versão antiga. Como não estou utilizando o anaconda tive que modificar um pouco os comandos para rodar:

```
None
py -m venv nome_do_ambiente #O padrão aqui geralmente é venv pois as IDEs já identificam que será um ambiente virtual

venv\Scripts\Activate #Ativa

# Aqui abaixo é o comando para caso querira mudar o python que sera utilizado
virtualvenv virtualenv -p caminho_do_diretorio #Coloca o caminho de onde a outra versão do python está instalada

pip install pandas numpy ... # para intalar dentro da venv sem precisar refatorar para testar algo

OBS: para o proximo card vou instalar o conda
```

Depois de ter aprendido a mexer no Jupyter Notebook, comecei a seção 5 *Numpy*, que é uma biblioteca de álgebra linear(que utiliza *arrays* vetoriais e matriciais) de alta eficiência ela tem sua base feita em linguagem C o que traz muito mais velocidade para as operações, para a instalação utilizei `pip install numpy`. Após a instalação aprendi como trabalhar com a manipulação de *arrays* abaixo deixei a tabela com os métodos vistos na aula:

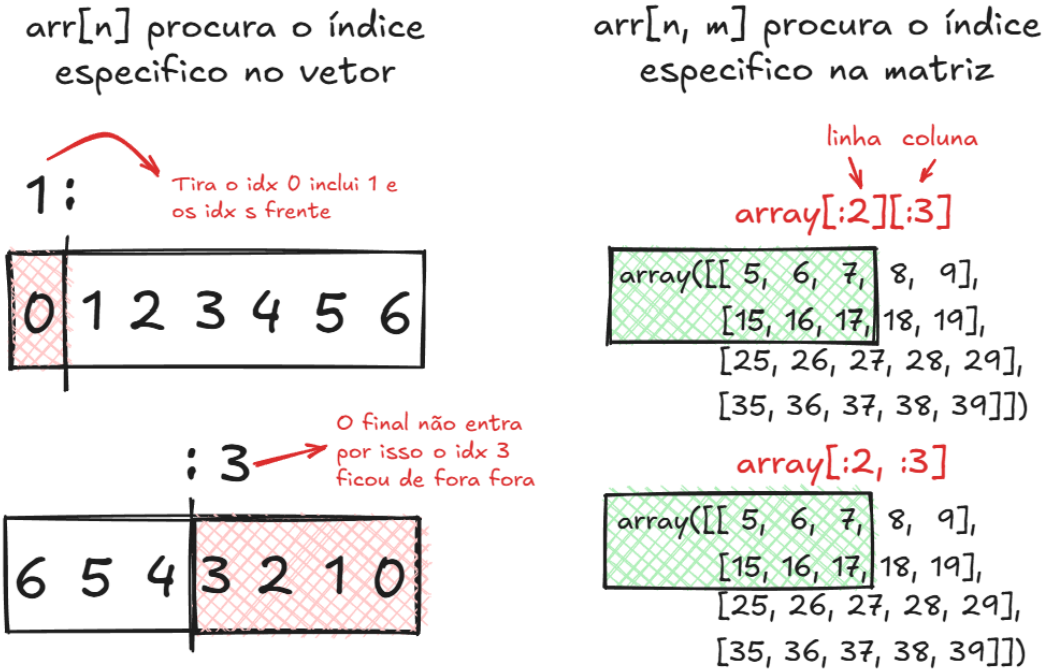
Tabela 1 - Principais métodos do *Numpy*.

Principais métodos	
.array()	Transformar em um array
.arange()	Cria um array com tam e step
.zeros()	Cria uma matriz de zeros
.ones()	Cria uma matriz de uns
.full()	Cria uma matriz de n(qualquer)
.linspace()	Pega a quantidade escolhida de valores de mantendo o esmo espaçamento entre eles
.reshape()	Redimensiona os elementos no shape escolhido
.max()	Devolve o maior valor do array
.min()	Devolve o menor valor do array
.argmax()	Devolve o índice do maior valor
.argmin()	Devolve o índice do menor valor
Sub-biblioteca Random	
.randint()	Sorteia um numero inteiro dentro do intervalo passado
.rand()	Sorteia um numero entre 0 e 1 e devolve com distribuição uniforme
.randn()	Sorteia um numero entre 0 e 1 e devolve com distribuição gaussiana

Fonte: autoria própria feita com o Excalidraw.

Depois na próxima aula aprendi como fazer fatiamento e aqui estão as formas de fatiamento:

Imagem 1 - Indexação e fatiamento no *Numpy*.



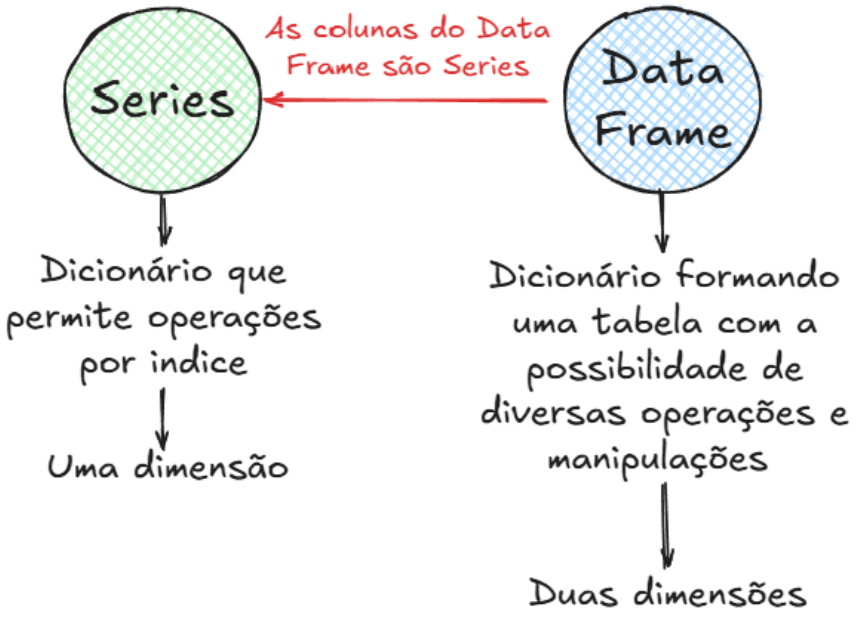
Fonte: autoria própria feita com o Excalidraw.

Um cuidado que deve-se tomar é a verificação de apontamento ou cópia, inicialmente sempre que se faz um fatiamento de um *array* no *Numpy* e atribui-se a uma variável *array* é feito um apontamento, portanto as alterações feitas nesse variável de *array* irão afetar diretamente o original, caso não queira isso evitar isso usa-se o `.copy()` que faz uma cópia e assim o *array* original não é alterado.

Na próxima aula, trabalhei com operações com números escalares e *arrays*, sendo elas multiplicação, soma, divisão(com exceções para casos envolvendo o zero na divisão sendo o `inf`,quando um *array* está dividindo algo e um dos elementos dele é zero e o outro caso é para quando um *array* ta dividindo outro sendo que no divisor dentro do *array* tem um elemento de valor zero a saída dessa operação é NaN). Por fim nessa seção resolvi alguns exercícios.

Terminada a seção 5 de *Numpy* fui para a seção 6 *Pandas*, que também é uma biblioteca do *Python,* inclusive ela foi escrita sobre o *Numpy*, ela nos permite trabalhar com data frames que são a principal forma de entrada e visualização de dados utilizada no aprendizado de máquina. Para a instalação utilizei o comando `pip install pandas`.

Imagem 2 - Tipos de estruturas de dados do *Pandas*.



Fonte: autoria própria feita com o Excalidraw.

Nas primeiras aulas desta seção foram apresentadas dois tipos de estruturas de dados a `Series` e o `DataFrame`(cada coluna é uma `Series`) que estão dispostos na imagem 2, depois foram apresentados os seguintes métodos para a criação, fatiamento, seleção e indexação dos Data Frames:

- `iloc()`: Forma de navegar pelos dados no mesmo formato do *Numpy*.
- `loc()`: Navegação através dos índices do `DataFrame`.
- `drop()`: método utilizado para fazer a deleção de uma linha ou coluna, nele você pode utilizar o `inplace` se colocar `=True` ele apaga do `DataFrame` original e se estiver com o `=False` ou o `default` ele apaga só na estrutura atual.
- `reset_index()`: redefine o índice atual para o índice padrão 0 1 2….
- `set_index()`: Permite transformar uma *series* do *data frame* em índice.

Além dos métodos acima, aprendi uma variação de indexação, o `MultiIndex` que permite fazer uma hierarquização dos níveis de índices como na imagem 3:

Imagem 3 - Exemplo de índice multinível..

Grupo	Numero		
G1	1	-1.278931	-0.362068
	2	-1.621847	1.318895
	3	-0.107222	-0.082291
G2	1	0.455042	0.771086
	2	0.146580	1.066209
	3	0.895303	-1.792451

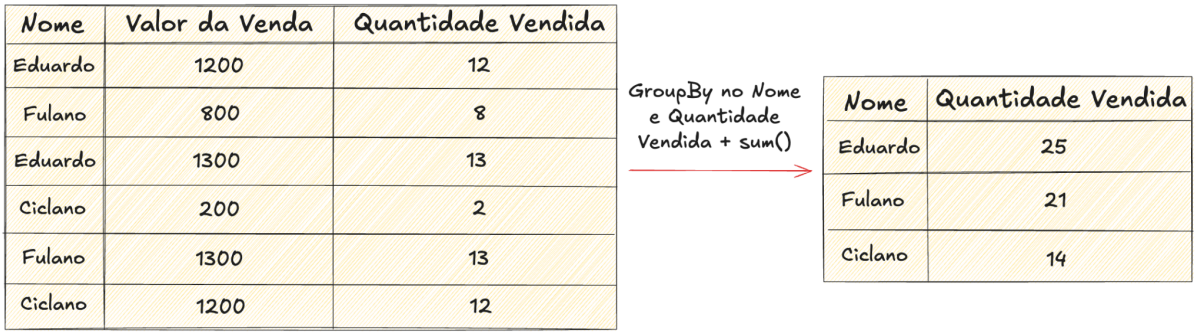
Fonte: autoria própria captura do Jupyter Notebook.

Depois no curso foi apresentado uma precaução para casos de dados ausentes que consiste na utilização de de algum métodos a fim de tratar os dados nan dispostos no data frame estes métodos seriam métodos seriam:

- `fillna()`: substituí os valores nan por algum valor passado.
- `dropna()`: esse elimina colunas ou linhas por completo caso possuam valores nan, podemos escolher através do axis= 0 ou 1 se queremos apagar linhas ou colunas ou os dois, também podemos utilizar o parâmetro que só apaga a linha ou coluna se tiver uma quantidade pré definida de valores nan, esse parâmetro é o thresh.
- `isnull()`: devolve um valor booleano para cada valor do dataframe.
- `nunique()`: devolve o tamanho do vetor de saída de dados únicos.
- `unique()`: retorna os valores únicos(mostra só uma vez os repetidos).

Além disso aprendi também a trabalhar com `groupby` que permite operações serem feitas comparando colunas por exemplo Nome com Quantidade Vendida como resultado vamos ter por exemplo Eduardo | 15, possibilitando diversas manipulações abaixo na Imagem 4 coloquei um exemplo.

Imagem 4 - Exemplo de GroupBy planilha de vendas.



Fonte: autoria própria feito no Excalidraw.

Depois disso aprendi a concatenar, cortar e mesclar. Para concatenar utiliza-se o método `concat()` podendo-se fazer a concatenação por linhas ou colunas isso é definido através do *axis,* uma ressalva é que quando vai ser feita uma concatenação é de extrema importância se atentar a disposição dos índices. Já no mesclar usa-se o método `merge()` que faz a mesclagem através das colunas podendo variar a forma como são mesclados(*inner, left, right e outer*). E o método do próprio dataframe não do pd join que possibilita a junção mas agora através do índice diferente do `merge()` que é através das colunas.

Por fim aprendi mais alguns conceitos, sendo eles operações e entrada e saída de dados. Na parte de operações conservaram-se muitas do *Numpy* por isso vou falar só das novas que foram o `apply()` que permite aplicar uma função ao *data frame,* outra operação foi a de `pivot_table()`, `sort_values()` e `value_counts()` que servem, respectivamente para agrupando o dataframe e aplicando operações inclusive índices multinível, ordenar uma coluna nos perfis ascendentes e descendente e verificar a frequência de valores em uma coluna. E na parte Entrada e saída de dados aprendi a manipular um *.csv*

3. Conclusão

O curso me ensinou a utilizar duas das principais bibliotecas para tratamento e operação com dados o Numpy e o Pandas, além disso também me ensinou a trabalhar com o ambiente virtual Jupyter que facilita muito o desenvolvimento por conta da divisão por células, células de markdown, compartilhamento e a execução interativa. Com esses conhecimentos adquiridos agora já posso começar a trabalhar com a parte de tratamento do dataset e lapidação de saídas no processo de aprendizado de um modelo.

4. Referências

TADEWALD, Rodrigo Soares. **Python para Data Science e Machine Learning – COMPLETO**. Udemy, [s.d.]. Disponível em: pasta do drive. Acesso em: 20 ago. 2026.